

P3492R1

Sized deallocation for placement new

Published Proposal, 2025-02-10

Author:

[Lauri Vasama](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:

vasama.github.io/wg21/D3492.html

Current draft source:

github.com/vasama/wg21/blob/main/D3492.bs

Abstract

Permit the selection of a sized placement deallocation function in new expressions.

Table of Contents

1 Introduction

- 1.1 Usual new and delete expressions
- 1.2 Placement new expressions
- 1.3 Usage with allocators

2 Proposal

- 2.1 Deduced parameters
- 2.2 Ambiguities
- 2.3 `std::align_val_t`

2.4 Feature-testing macro

3 Alternatives

3.1 Library function

4 Implementation experience

5 Acknowledgements

References

Informative References

§ 1. Introduction

§ 1.1. Usual new and delete expressions

NOTE: For simplicity these examples ignore any class specific allocation or deallocation functions.

Given the new expression `new T`, the compiler selects an allocation function matching one of the following calls:

- `operator new(sizeof(T))`
- `operator new(sizeof(T), std::align_val_t(alignof(T)))`

Given the delete expression `delete t`, the compiler selects a deallocation function overload matching one of the following calls:

- `operator delete(storage_ptr)`
- `operator delete(storage_ptr, std::align_val_t(alignof(T)))`
- `operator delete(storage_ptr, sizeof(T))`
- `operator delete(storage_ptr, sizeof(T), std::align_val_t(alignof(T)))`

When deallocation functions both with and without size parameters are present, it is unspecified which is selected.

NOTE: In both cases the preference for passing an alignment value depends on whether `T` has new-extended alignment.

Going back to the case of `new T`; if the initialisation of the `T` object were to throw an exception, a *matching* deallocation function is used to deallocate the previously allocated storage. Currently the wording on this is not very clear, but it can be assumed that the intent is to use the deallocation function selected by `delete T`. That might mean selecting a deallocation function not exactly matching the parameters of the used allocation function. For example:

- `operator new(sizeof(T))`
- `operator delete(storage_ptr, sizeof(T))`

Here an additional size parameter is passed to the deallocation function.

§ 1.2. Placement new expressions

Given the placement new expression `new (args...) T`, the compiler selects an allocation function matching one of the following calls:

- `operator new(sizeof(T), args...)`
- `operator new(sizeof(T), std::align_val_t(alignof(T)), args...)`

If the initialisation of the `T` object were to throw an exception now, the storage may be deallocated *if* a matching placement deallocation function is found. In this case a deallocation function matches the allocation function if its parameters, after parameter transformations, are all identical to the parameters of the selected placement allocation function, except for the first parameter which is to be `void*`. It is notable that in the case of failed placement new expressions, no deallocation function containing a size parameter will ever be selected. This is unfortunate for the same reasons explained in [\[N3778\]](#) which introduced the global sized deallocation functions.

Furthermore, unlike global deallocation functions, placement allocation functions are very intentionally provided context for the allocation. Since user allocation schemes are much less constrained than the global allocation and deallocation functions, it is conceivable that a user allocator providing a placement allocation function for ease use might be unable to deallocate the memory without being explicitly provided the size of the allocation. In fact this proposal was created after encountering just that scenario.

NOTE: The terms placement allocation function and placement deallocation function are not currently defined, but we take them to mean that set of allocation functions which are only ever selected by placement new expressions and not by usual new expressions, and the matching set of deallocation functions. See [\[CWG2592\]](#).

§ 1.3. Usage with allocators

The following pattern, using a custom placement allocation function is somewhat common:

```
T* ptr = new (alloc) T(args...);
```

If the construction throws an exception, and a matching placement deallocation function exists, it is invoked to free the allocated memory. That deallocation function does not have access to the size of the allocation. Therefore if `alloc` were to follow the standard allocator model, it would not be possible to deallocate the memory, because the standard allocator model requires providing the size of the allocation to its `deallocate` function.

§ 2. Proposal

We propose to permit placement new expressions to use two partially matching deallocation functions:

- `operator delete(storage_ptr, std::size_val_t(sizeof(T)), args...)`
- `operator delete(storage_ptr, std::size_val_t(sizeof(T)),
std::align_val_t(alignof(T)), args...)`

For the backwards compatibility reasons explained in [\[P0035R4\]](#), which introduced `std::align_val_t`, we propose to use a similar type with the name `std::size_val_t` for passing the size of the allocation. This in conjunction with ignoring overloads with deduced `std::size_val_t` parameters prevents changing the meaning of any existing code.

```
namespace std {  
    enum class size_val_t : size_t {};  
}
```

Any matching overloads using `std::size_val_t`, if available, are used in preference to those existing ones without `std::size_val_t`. This is because a deallocation function with access to the size is no less efficient than one without, and thanks to the new `std::size_val_t`, no existing code should have its meaning changed. In addition, the use of `std::size_val_t` allows opting into new safer behaviour where ambiguities are no longer silently ignored.

§ 2.1. Deduced parameters

In order to avoid matching existing placement allocation function *templates*, no otherwise matching overload is considered if the type of the `std::size_val_t` parameter was deduced.

Neither of these functions are valid candidates for sized placement deallocation:

```
void operator delete(void* s, auto...);  
void operator delete(void* s, std::same_as<std::size_val_t> auto s, my_allocator& a);
```

§ 2.2. Ambiguities

In order to avoid accidental memory leaks, if more than one matching placement allocation function with a `std::size_val_t` is found the program is ill-formed.

```
void operator delete(void* p, std::size_val_t s, my_allocator& a, long);  
void operator delete(void* p, std::size_val_t s, my_allocator& a, long long);  
  
new (alloc, 42) T; // Ill-formed
```

This is in contrast to existing placement new expressions, where ambiguous placement deallocation functions are simply ignored, as if no matching function was found at all.

§ 2.3. `std::align_val_t`

If the selected placement allocation function contains an implicit `std::align_val_t` parameter, as per the current rules, a matching placement deallocation function must also contain such a parameter. In this case the new `std::size_val_t` parameter is placed directly after the `void*` parameter and before the `std::align_val_t` parameter if any.

```
void* operator new(size_t s, std::align_val_t a, my_allocator& al);  
  
// The only currently matching overload:  
void operator delete(void* p, std::align_val_t a, my_allocator& al);  
  
// This proposal allows this overload to be matched:
```

```
void operator delete(void* p, std::size_val_t s, std::align_val_t a, my_al`

// This overload is never used:
void operator delete(void* p, std::align_val_t a, std::size_val_t s, my_al`
```

§ 2.4. Feature-testing macro

Add the following feature test macro: `__cpp_sized_placement_deallocation`



§ 3. Alternatives

§ 3.1. Library function

Any discussion of placement new and its deallocation behaviour raises the obvious question of placement deletion. Suppose one uses the placement new syntax to create objects of dynamic storage duration using some custom allocator: `new (alloc) T`. How does one then delete those objects? If explicit deletion is needed at all, most likely that is achieved using a function template taking the allocator and a pointer to the object to be deleted:

```
template<typename Allocator, typename T>
void delete_via(Allocator const& allocator, T* const ptr)
{
    std::destroy_at(ptr);
    allocator.deallocate(ptr, sizeof(T));
}

delete_via(alloc, new (alloc) T);
```

Or possibly by a more general function template implementing placement delete via calls to placement deallocation functions:

```
delete_via(alloc)( new (alloc) T );
```

Why then, should we not also use a function template for the object creation instead of placement new?

```
template<typename T, typename Allocator, typename... Args>
    requires std::constructible_from<T, Args...>
T* new_via(Allocator const& allocator, Args&&... args);

// new (alloc) T(a, b, c);
new_via<T>(alloc, a, b, c);
```

That is indeed possible and allows one to solve the problem in library, but there are some major drawbacks:

1. Syntax

The placement new syntax provides direct syntactic access to the object initialisation, which has real benefits:

- An `emplace`-like function tends to lose IDE hints pertaining to the constructors of the type `T` being initialised.
- Directly initialising the object allows for more expressive forms of initialisation, such as designated initialisation:

```
new (alloc) T{ .a = x, .b = y }
```

- The initialisation of `T` requires access to its constructors. If those constructors are not publicly accessible, `new_via` does not have access to them, unless befriended. Direct use of placement new has no such limitation.

These things can still of course be achieved in library using a lambda, but this further degrades the user experience:

```
new_from_result_of(alloc, [&]() { return T{ .a = x, .b = y }; })
```

2. Existing usage

The placement new syntax already exists and is widely used. Instead of inventing a new library alternative, we can improve the performance of code already out there and enable the same code to work correctly with new kinds of user-defined allocators.

§ 4. Implementation experience

This proposal was implemented in Clang at github.com/vasama/llvm (branch P3492).

This implementation was tested against large real world code bases that make extensive use of placement allocation functions and function templates, including function templates containing deduced parameter packs.

§ 5. Acknowledgements

Big thanks to Oliver Hunt for the help.

§ References

§ Informative References

[CWG2592]

Jim X. *Missing definition for placement allocation/deallocation function*. 14 April 2022. open.
URL: <https://wg21.link/cwg2592>

[N3778]

Lawrence Crowl. *C++ Sized Deallocation*. 27 September 2013. URL: <https://wg21.link/n3778>

[P0035R4]

Clark Nelson. *Dynamic memory allocation for over-aligned data*. 21 June 2016. URL:
<https://wg21.link/p0035r4>